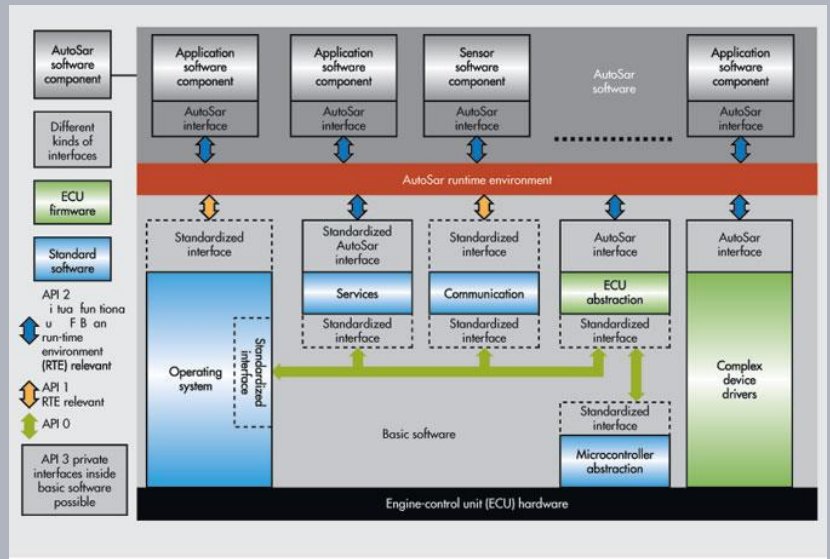
Advanced Microcontroller Architectures and Embedded OS

Autosar / RTE Intro

Prof. Dr.-Ing. Peter Fromm

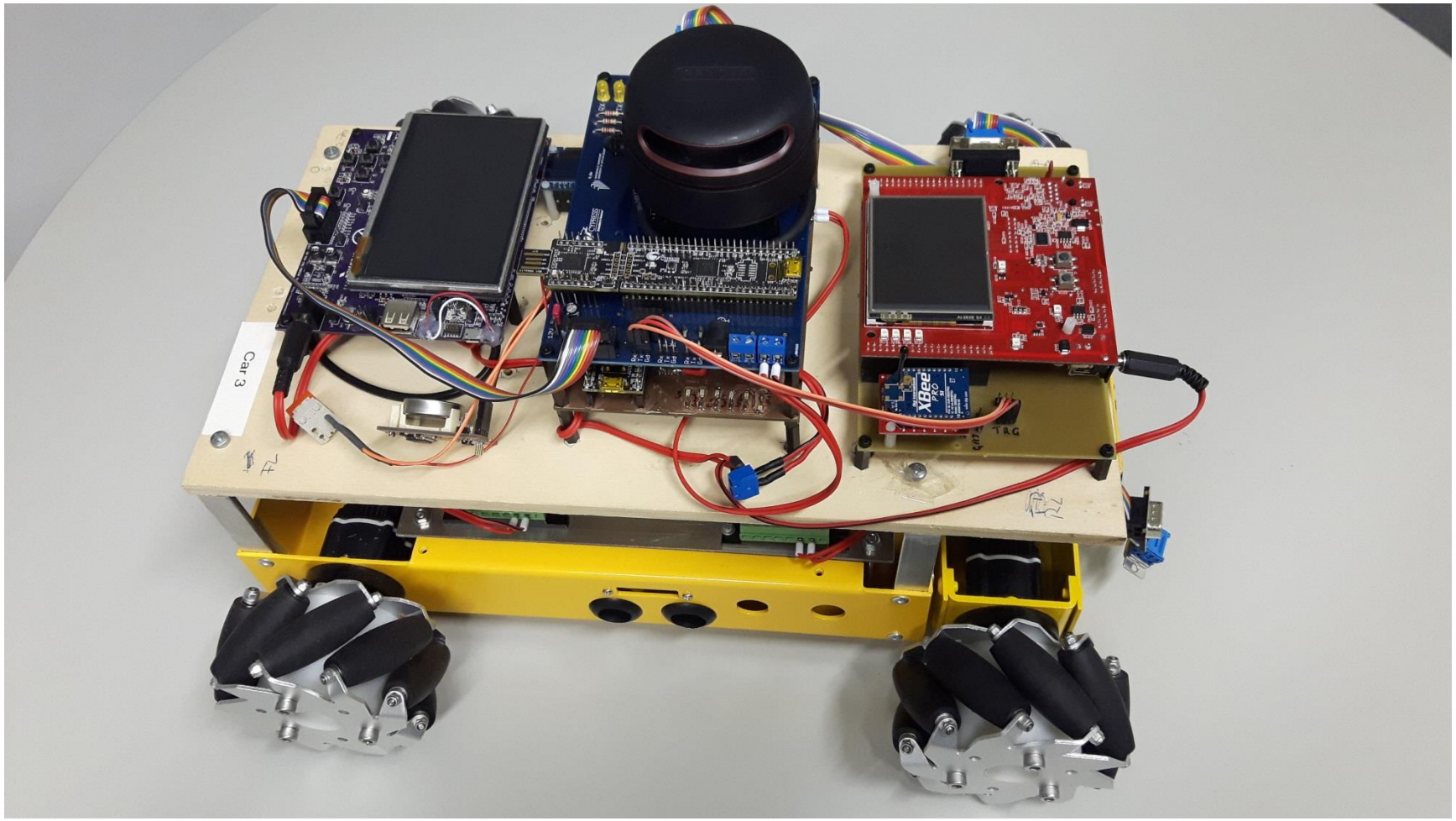


The AutoSar architecture covers many of the protocols necessary for automotive infotainment.

Content

- Autosar
- Development Workflow
- Virtual Function Bus – RTE
- Application Design
- System Integration
- Simplified Concept – RTE Light

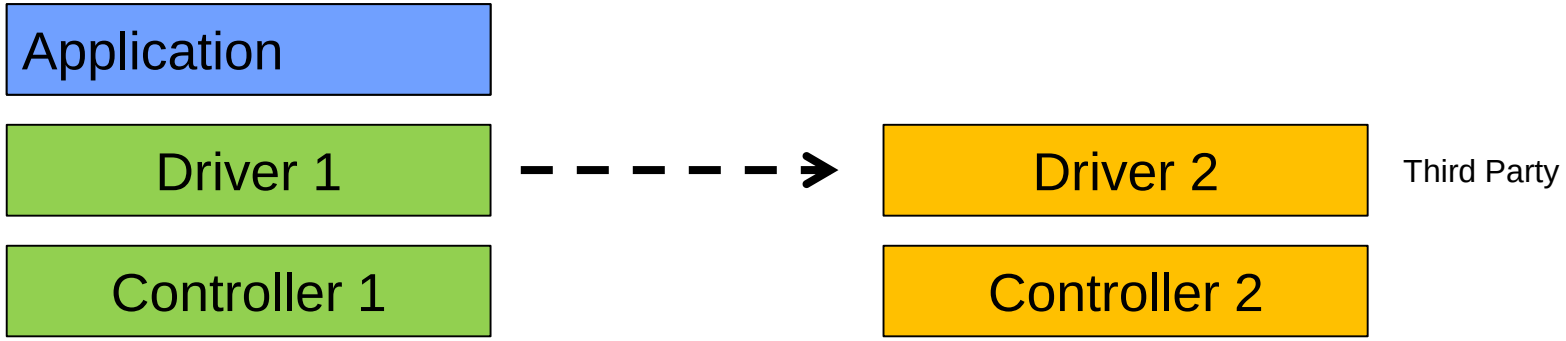
Let’s imagine...



Autosar concept

General design concept

- Focus on application development
- Exchange / reuse of components
- Strict separation of driver software (BSW) and application software (SWC)



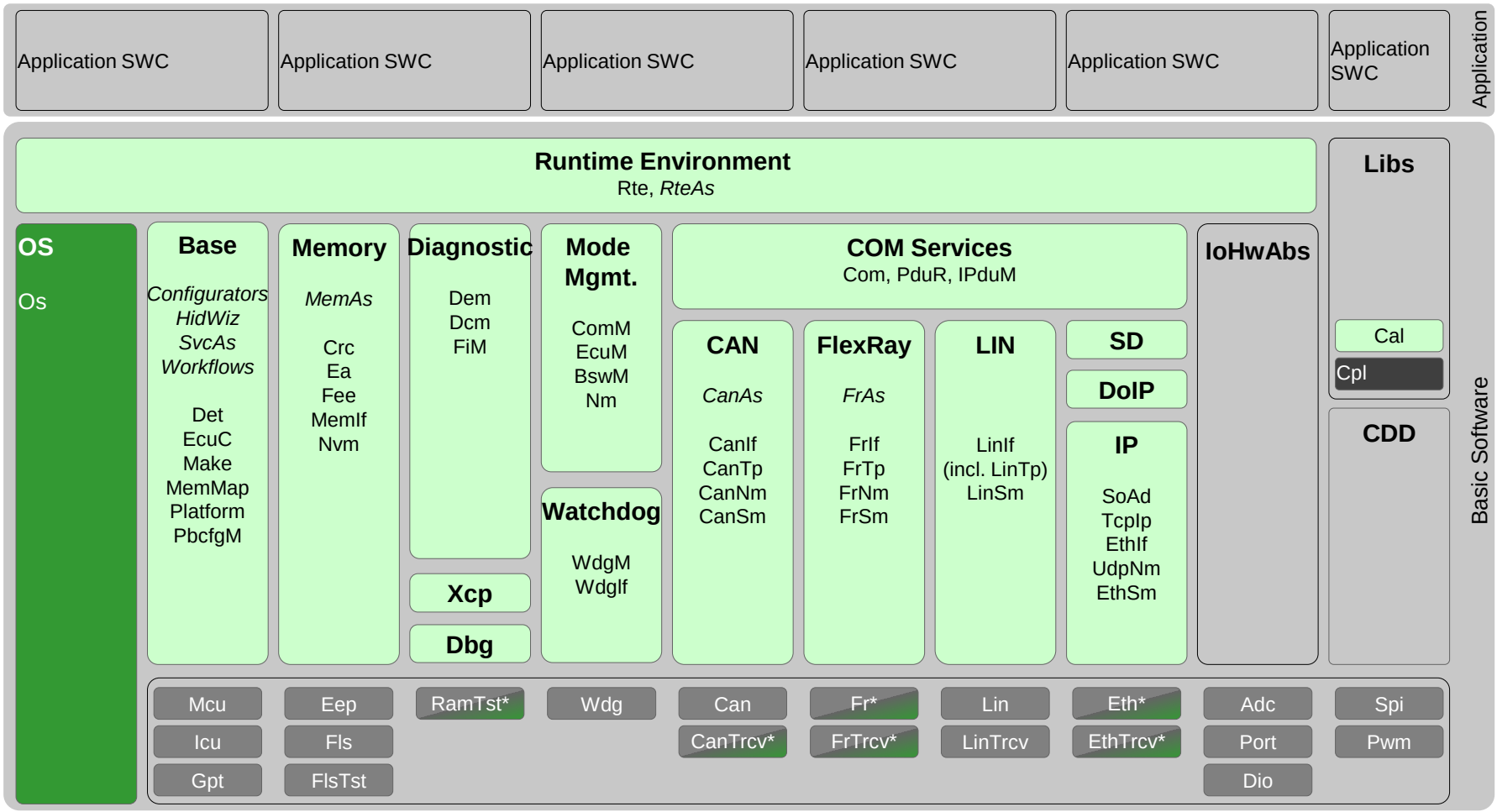
Additional motivation

- Often, the hardware has to be developped parallel with the software
- Debugging on a PC is simpler (and cheaper) compared to an embedded target
- Model based development supports the development process (especially in contral based applications)

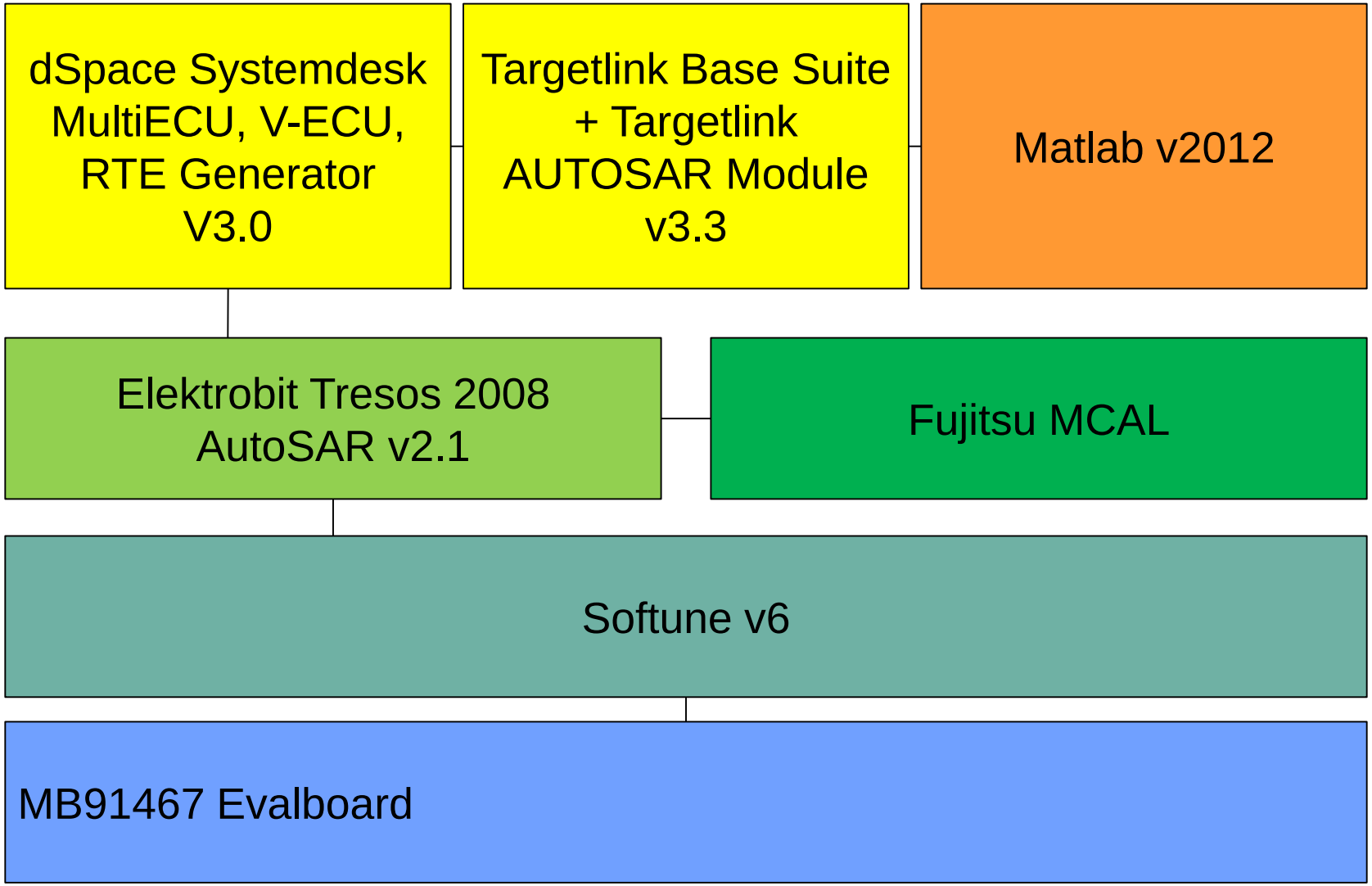
Need for hardware abstraction.

Sounds easy, but how?

Autosar Architecture



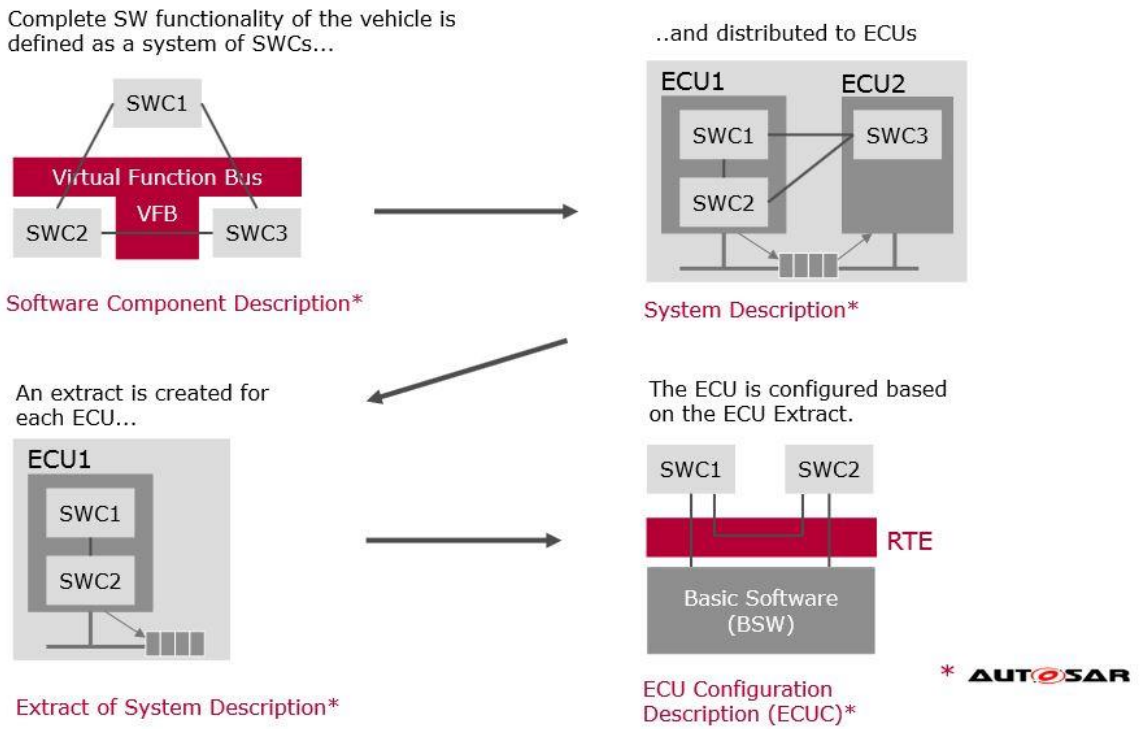
Autosar Toolchain (Sample)



Workflow

OEM, e.g. BMW

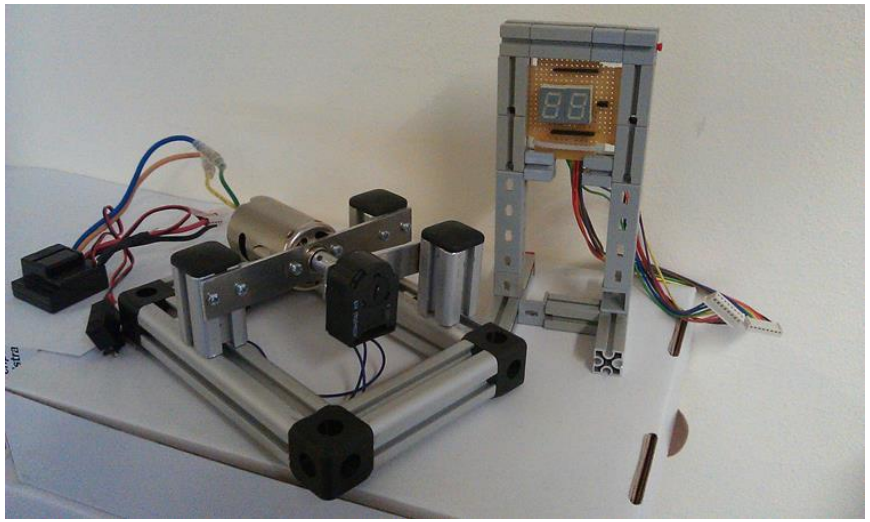
TIER1, e.g. Bosch



source: Autosar

Example – Control System

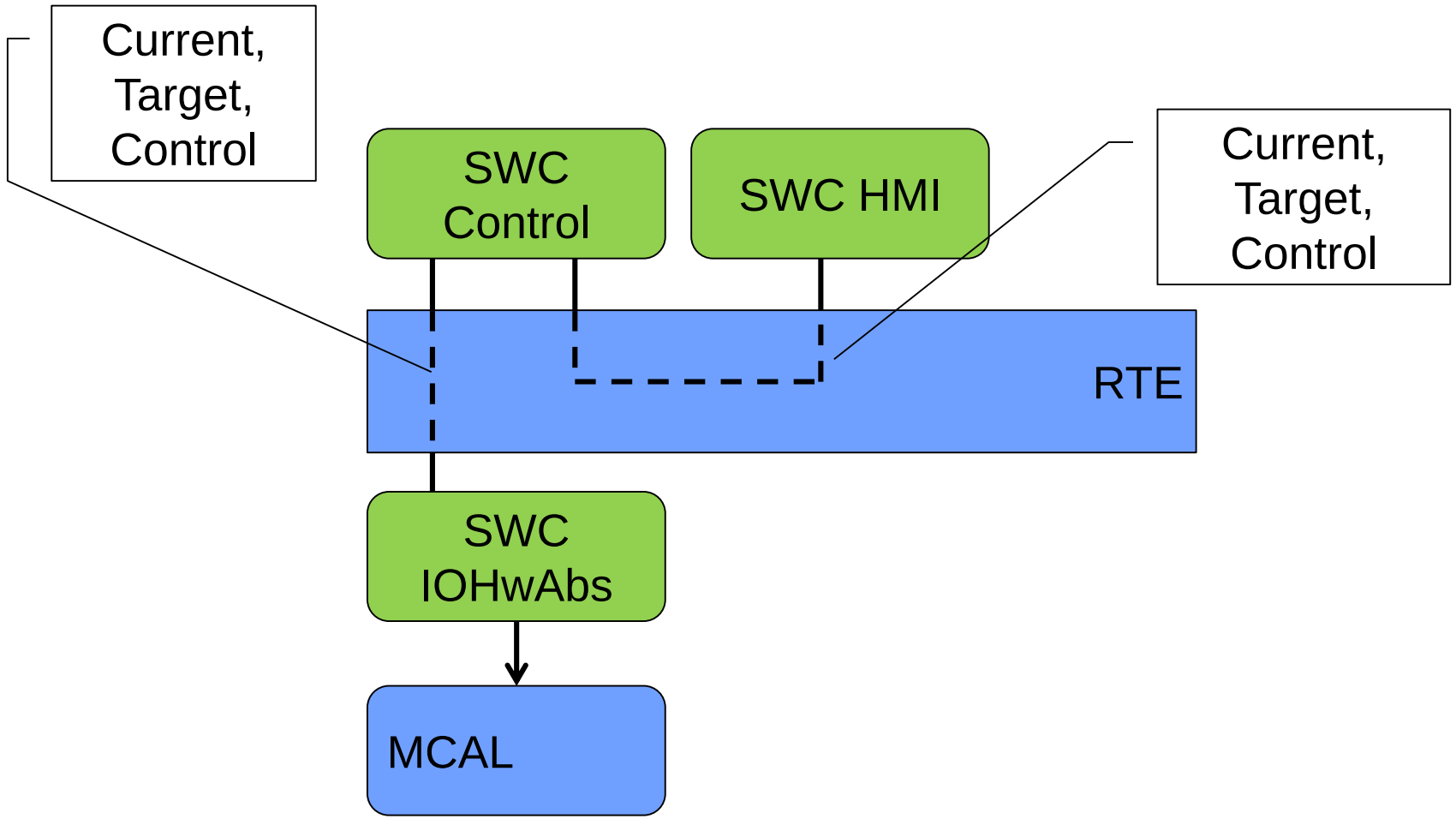
- Target Speed is read from a potentiomenter
- Current speed is read from an incremental decoder
- Control speed is calculated by controller
- The different parameters can be displayed on a 7-segment display
- A button will be used to toggle between the different values to be displayed



First iteration – software components and data flow

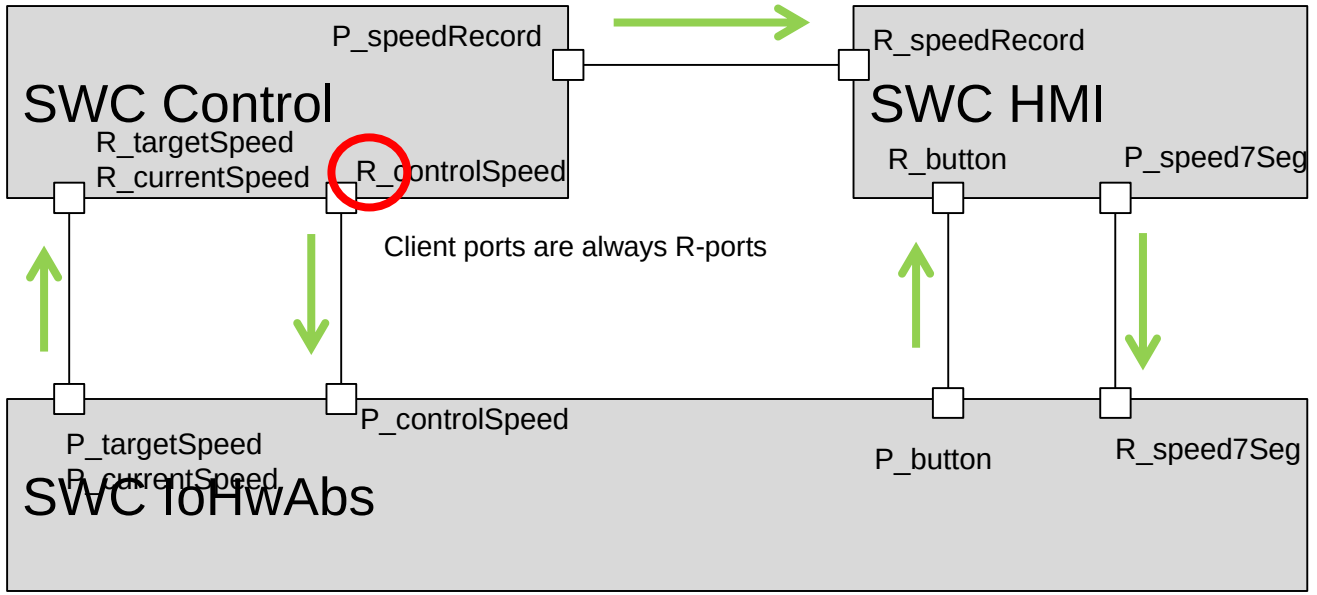
- A software component represents a certain functionality
- Ports are used to represent the external interfaces for the components
- The component IoHwAbs provides access to all peripherals (basic software - MCAL)
- Provide ports will provide data, Require ports will require data

Concept of the Runtime Environment / Virtual Function Bus



SWC = Software Component

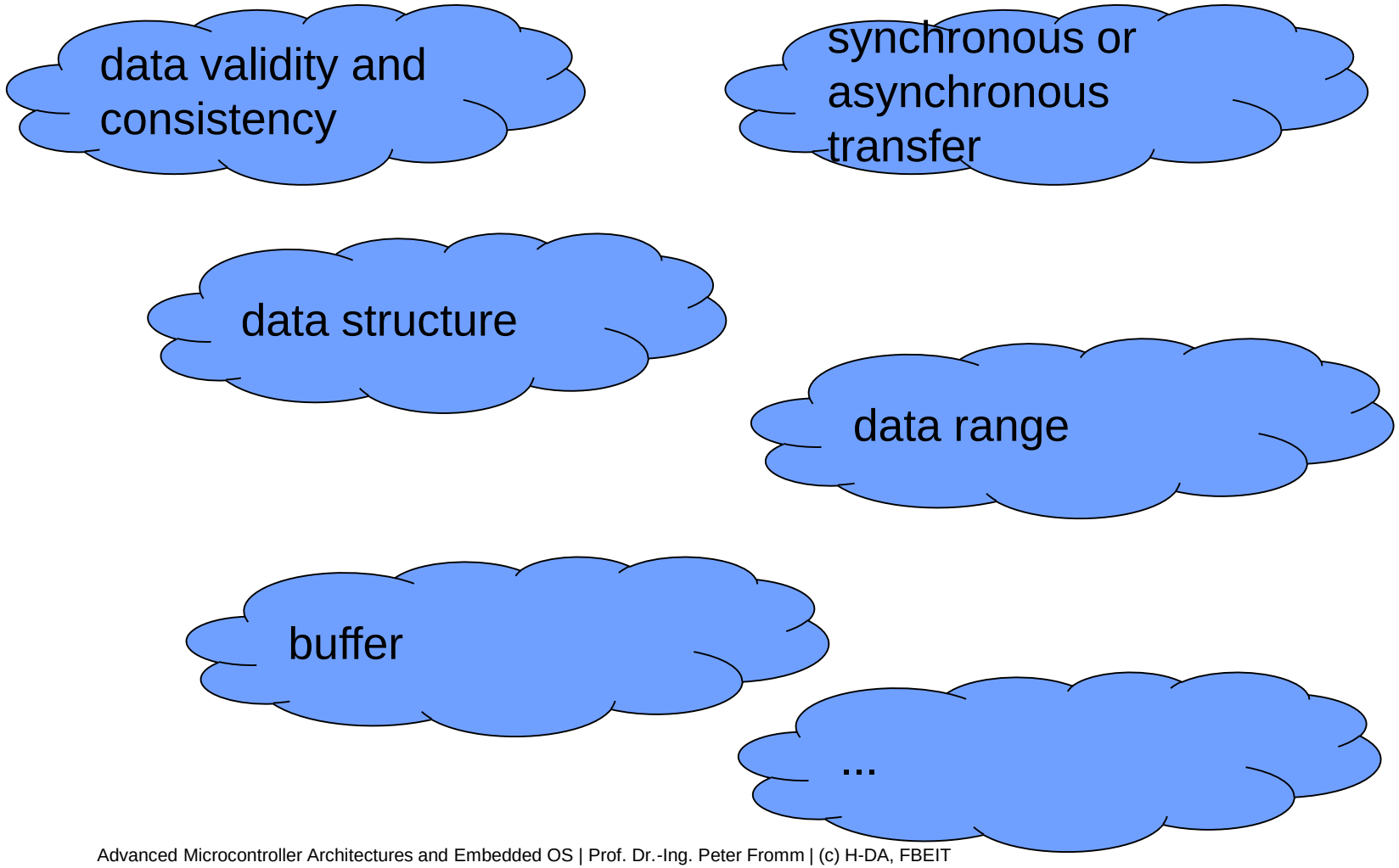
A first high level design



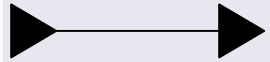
Require Ports require a service or data
Provide port provide a service or data

Second Iteration – data representation and interface design

A port is a very abstract description of the interface. What do we need to describe the interface in a more technical way?



Interface types

	Client / Server	Sender / Receiver
Type	• Synchronous	• Asynchronous
Implementation	• Function call	• Copy data to central buffer • Fire event
Features	• Un-queued • Queued	• Timeout • Resync
Symbol		
Execution order	• Explicit (event is fired directly after data transmission) • Implicit (event is fired when all data for a runnable is transferred)	

When to use which interface?

When to use which interface?

Client/Server

- When the execution order is crucial
- For short function calls
- Example: Getter/Setter for a port

Sender/Receiver

- When the execution time of the receiver is independant from the sender
- For broadcasting operations
- Example: A new dataset is provided from the COM-stack

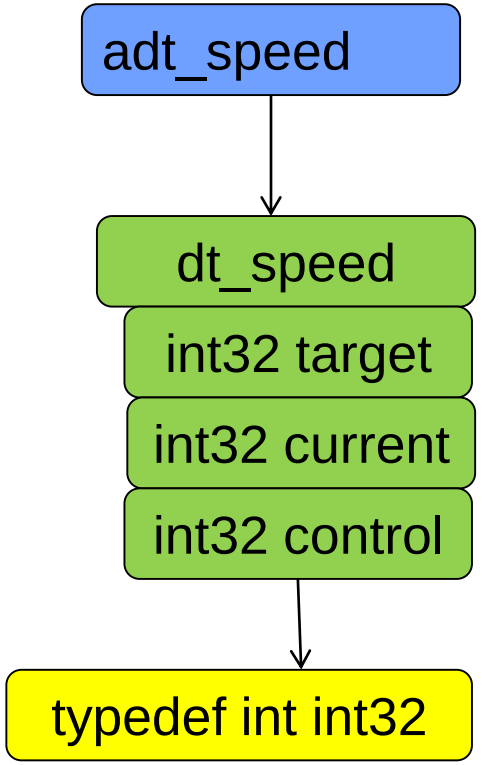
Data Types

Autosar provides several levels of abstraction for data types:

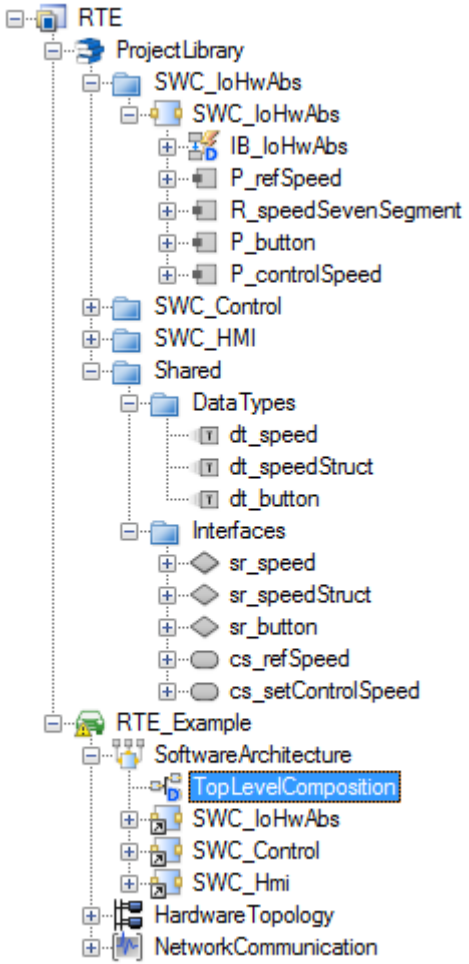
Application data types
- Used for the application design
- Implementation independent

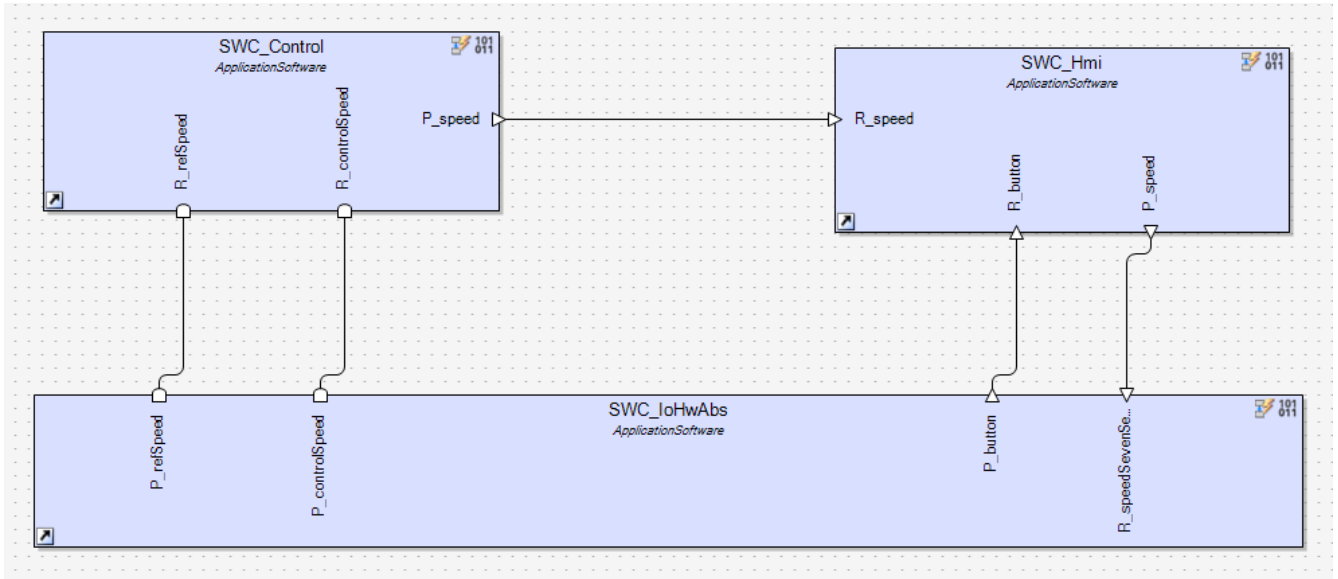
Implementation data types
- Used for the implementation
- Application data types have to be mapped to implementation data types

Machine data types
- Atomic data types like sint32, int32 are mapped to compiler data types
- like int, unsigned int



Model for our control system – external behavior

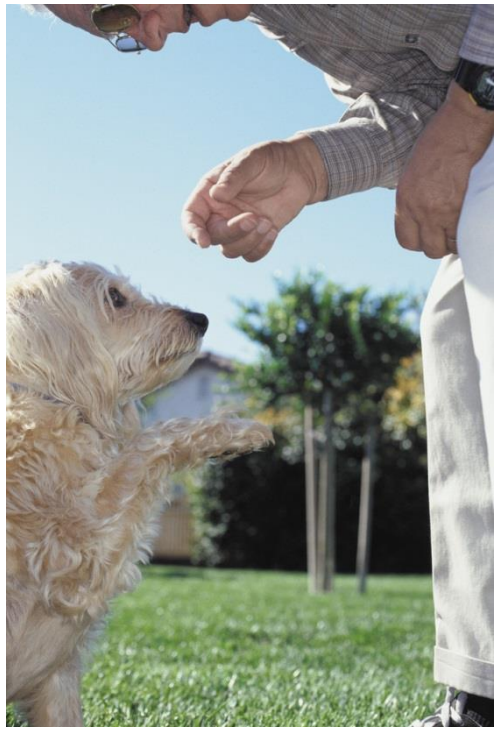




Checkpoint – What do we have?

Until now, we have described the external interfaces of our system components.

- At this level, we can give every component to one developer and ask him to implement this component.
- As long as nobody messes around with the agreed interfaces, this concept should work fine. Or not?



Internal Behavior

Every software component contains an internal behavior, which consists of

- 1..n Runnables
- RTE Events
- 1 Implementation

Runnable

- Comparable to a task
- Contains a certain functionality and reacts on RTE events, like incoming data, mode changes, time triggers,...

RTE Events

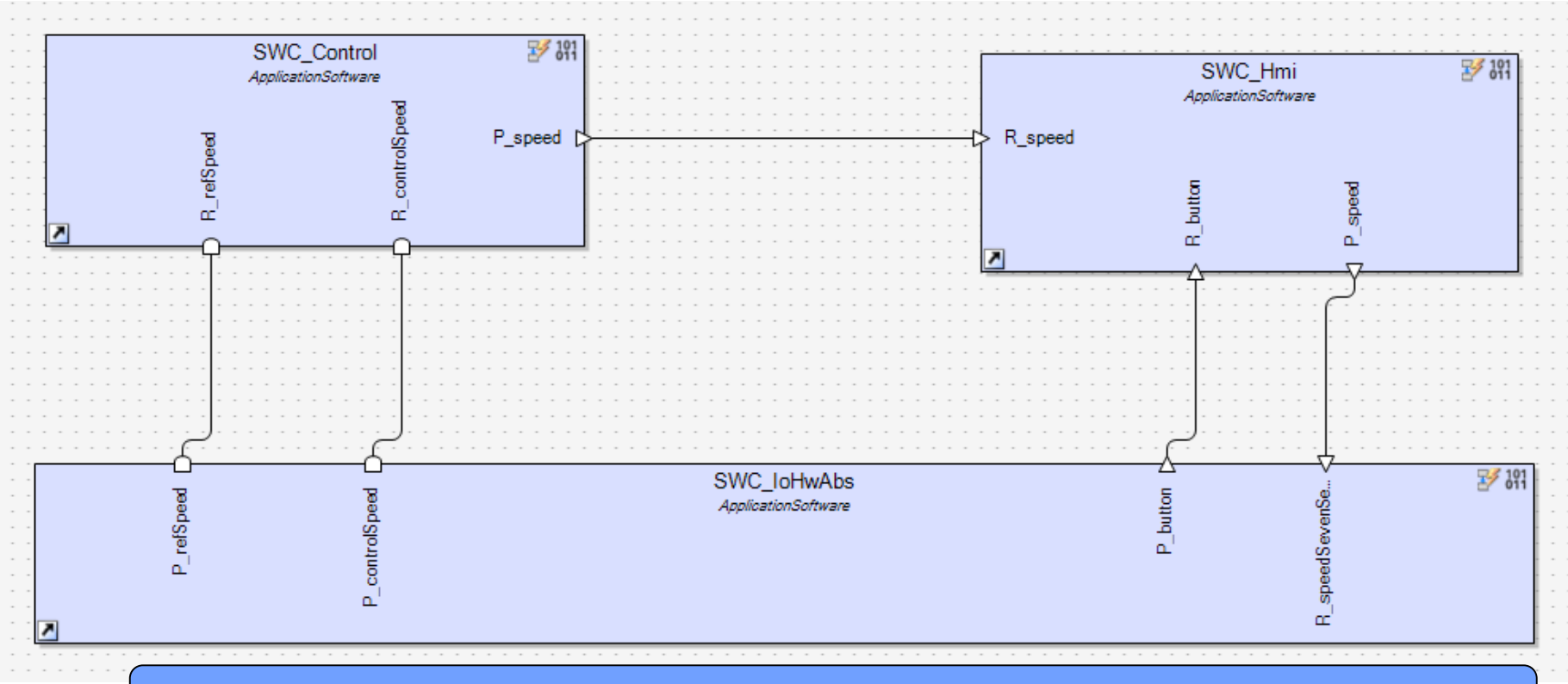
- RTE events are used to trigger the runnables

Implementation

- Information for the build process
- Name of the generated source files, compiler name,...

Runnables

- Runnables are the implementation units in an Autosar architecture
- A runnable can contain one or more functions
- A runnable will be called by events

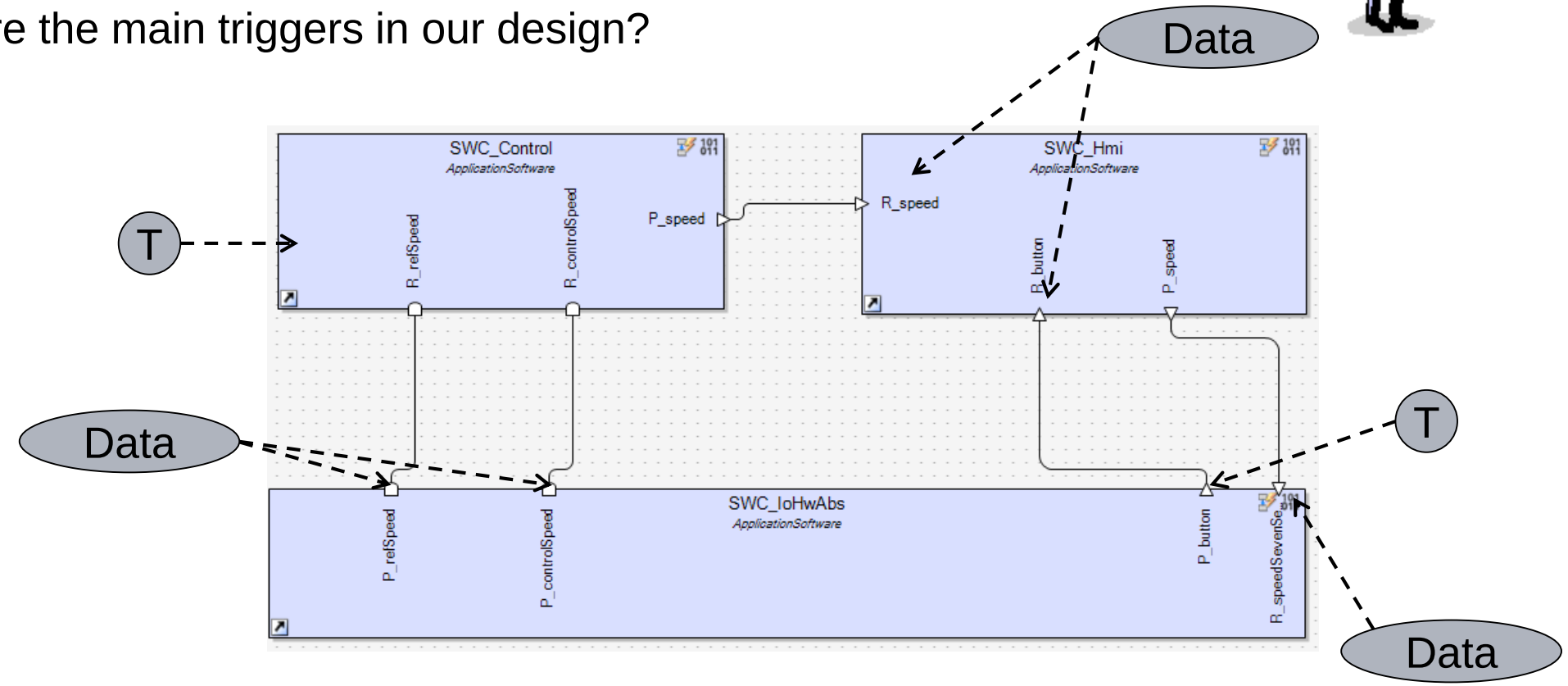


Which runnables do we need?

RTE Events

Problem: If everybody starts to define triggers as they like, the integration of the system will fail.

What are the main triggers in our design?



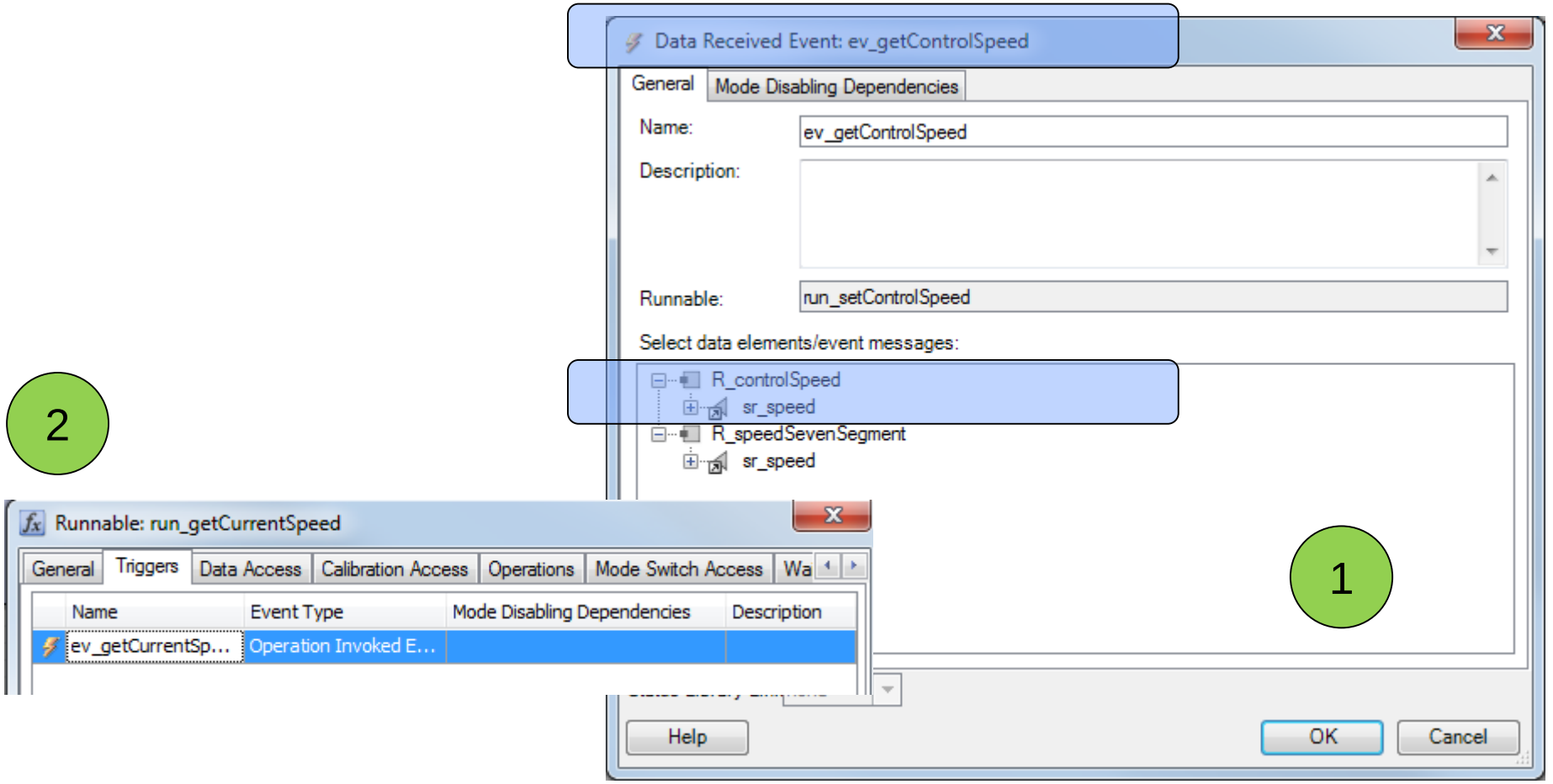
RTE Events

- The runnable control will be triggered by a timing event
  - The runnable will request data from the IOHwAbs by an invokeOperation call
  - The runnables in IoHwAbs will be triggered by the invokeOperation events
  - ...

Recipe: Try to find the master event and from this point derive the port related events. Then connect every runnable to at least one event.

RTE Events

- The event has to be defined in the internal behavior of the component
- And added to the runnable trigger list



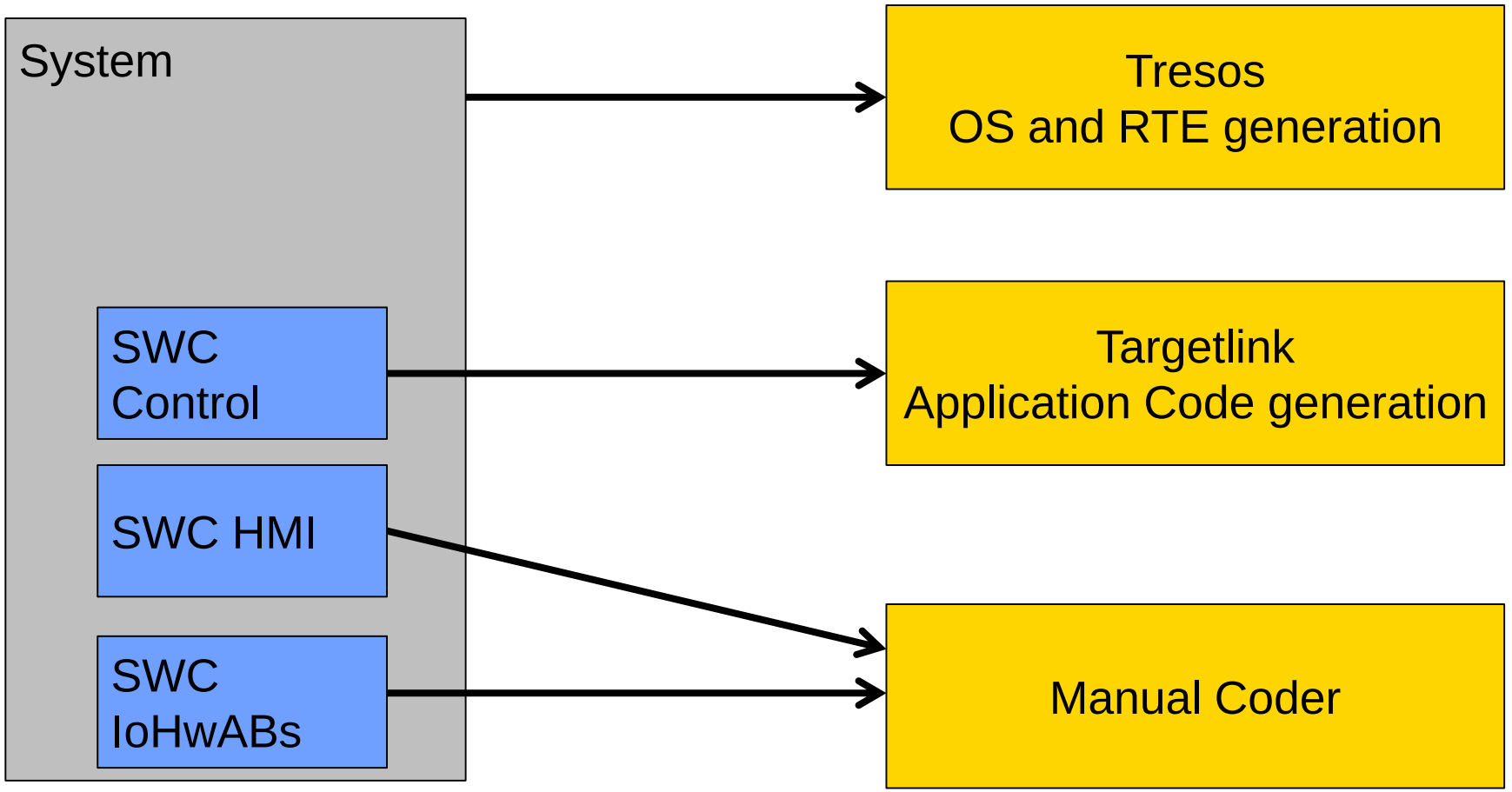
Checkpoint – What do we have?

- The component structure has been defined
- The interfaces for every component have been specified
- The “timeline” has been specified using RTE events
- The internal task structure has been defined using runnables
- All central data types are clearly specified

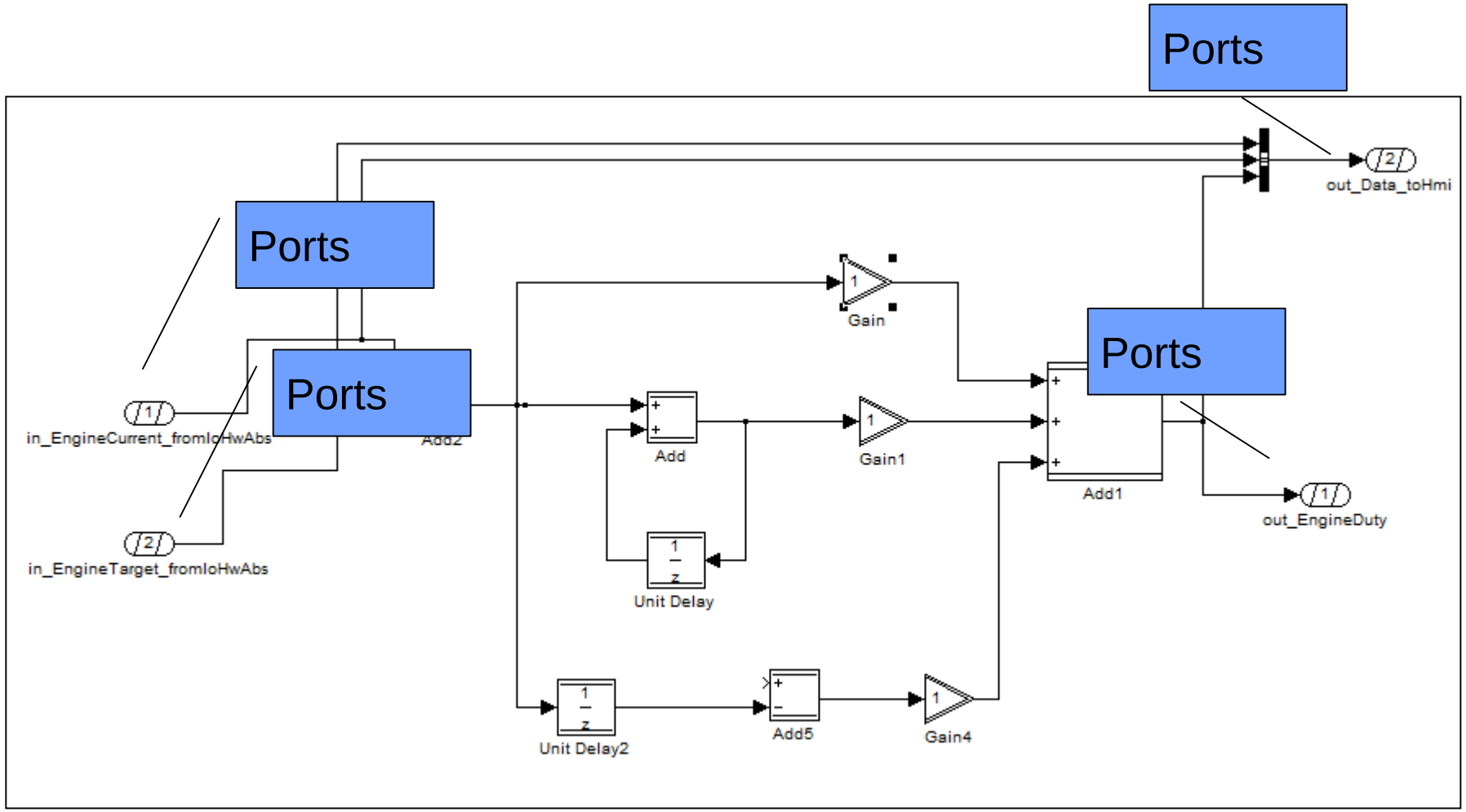
- Now the coding starts...

Export to Autosar XML

As a part of the Autosar specification, a XML schema has been defined to exchange data between the various tools in an Autosar project.



Example: Targetlink



Example: Autocode of a PID controller

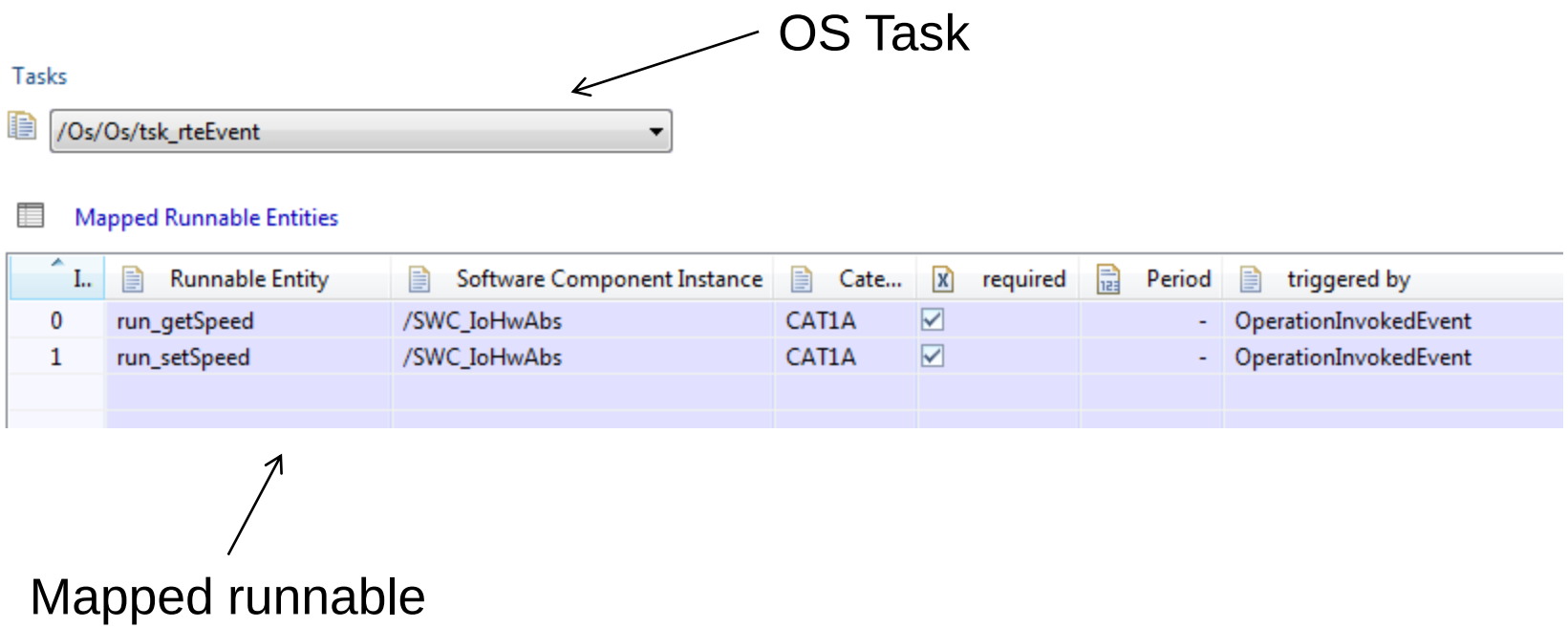
```
FUNC(void, RTE_APPL_CODE) SWC_Control_run_control(void)
{
   /* SLLocal: Default storage class for local variables | Width: 32 */
   dt_speed currentSpeed;
   dt_speed targetSpeed;

   /* SLLocal: Default storage class for local variables | Width: 16 */
   sint16 Sa4_Sum;
   sint16 Sa4_Sum2;
   sint16 Aux_S16;

   /* SLStaticLocalInit: Default storage class for static local variables with initvalue | Width: 16
    */
   static sint16 X_Sa4_Unit_Delay = 0;
   static sint16 X_Sa4_Unit_Delay1 = 0;

   /* BusInport: control_loop/SWC_Control/run_control/OA_R_refSpeed_getRefSpeed */
   Rte_Call_R_refSpeed_getRefSpeed(&targetSpeed, &currentSpeed);

   /* Sum: control_loop/SWC_Control/run_control/Sum */
   Sa4_Sum = (sint16) (((uint16) targetSpeed) - ((uint16) currentSpeed));

   /* Sum: control_loop/SWC_Control/run_control/Sum2 */
   Sa4_Sum2 = (sint16) (((uint16) Sa4_Sum) + ((uint16) X_Sa4_Unit_Delay));

   /* Unit delay: control_loop/SWC_Control/run_control/Unit Delay */
   X_Sa4_Unit_Delay = Sa4_Sum2;

   /* Sum: control_loop/SWC_Control/run_control/Sum1
      # combined # Gain: control_loop/SWC_Control/run_control/Kp */
   Aux_S16 = Sa4_Sum;

   /* # combined # Gain: control_loop/SWC_Control/run_control/Ki */
   Aux_S16 += Sa4_Sum2;

   /* # combined # TargetLink outport: control_loop/SWC_Control/run_control/OA_R_controlSpeed_setCon
      trolSpeed
      # combined # Gain: control_loop/SWC_Control/run_control/Scaling
      # combined # Gain: control_loop/SWC_Control/run_control/Kd
      # combined # Sum: control_loop/SWC_Control/run_control/Sum3 */
   Rte_Call_R_controlSpeed_setControlSpeed((sint32) (sint16) ((((sint32) (sint16) (((uint16)
    Aux_S16) + ((uint16) (sint16) (((uint16) Sa4_Sum) - ((uint16) X_Sa4_Unit_Delay1))))) * 21845) >>
     16));

   /* Unit delay: control_loop/SWC_Control/run_control/Unit Delay1 */
   X_Sa4_Unit_Delay1 = Sa4_Sum;
}
```

Example: Manual Coding of a runnable

```
void IOHW_setLed (void)
{

Std_ReturnType status = E_OK;

  dt_port8 value;

  status = Rte_Read_in_LedValue_value(&value);



  //Application Code
  Dio_WritePort(DioPort_4,value);
  //End Application Code

}
```

System Generation

Now, the runnables have to be mapped to OS tasks:



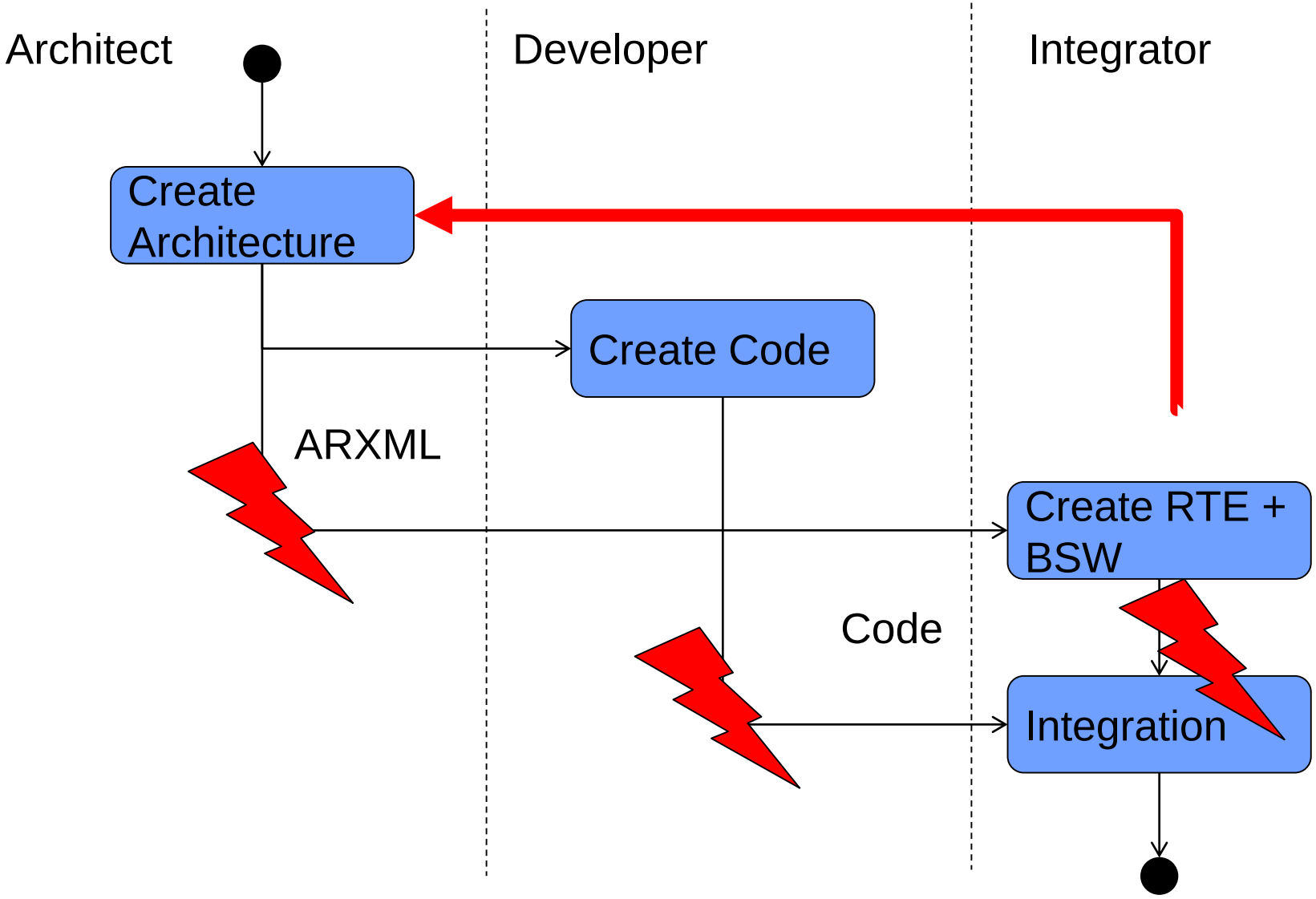
Tresos – Generation of OS

- Mapping of runnables to tasks
  - Time triggered runnables to a time triggered task
  - Event triggered runnables to an event triggered task
- Protection of critical sections
  - Enable/disable IRQ
  - Resources

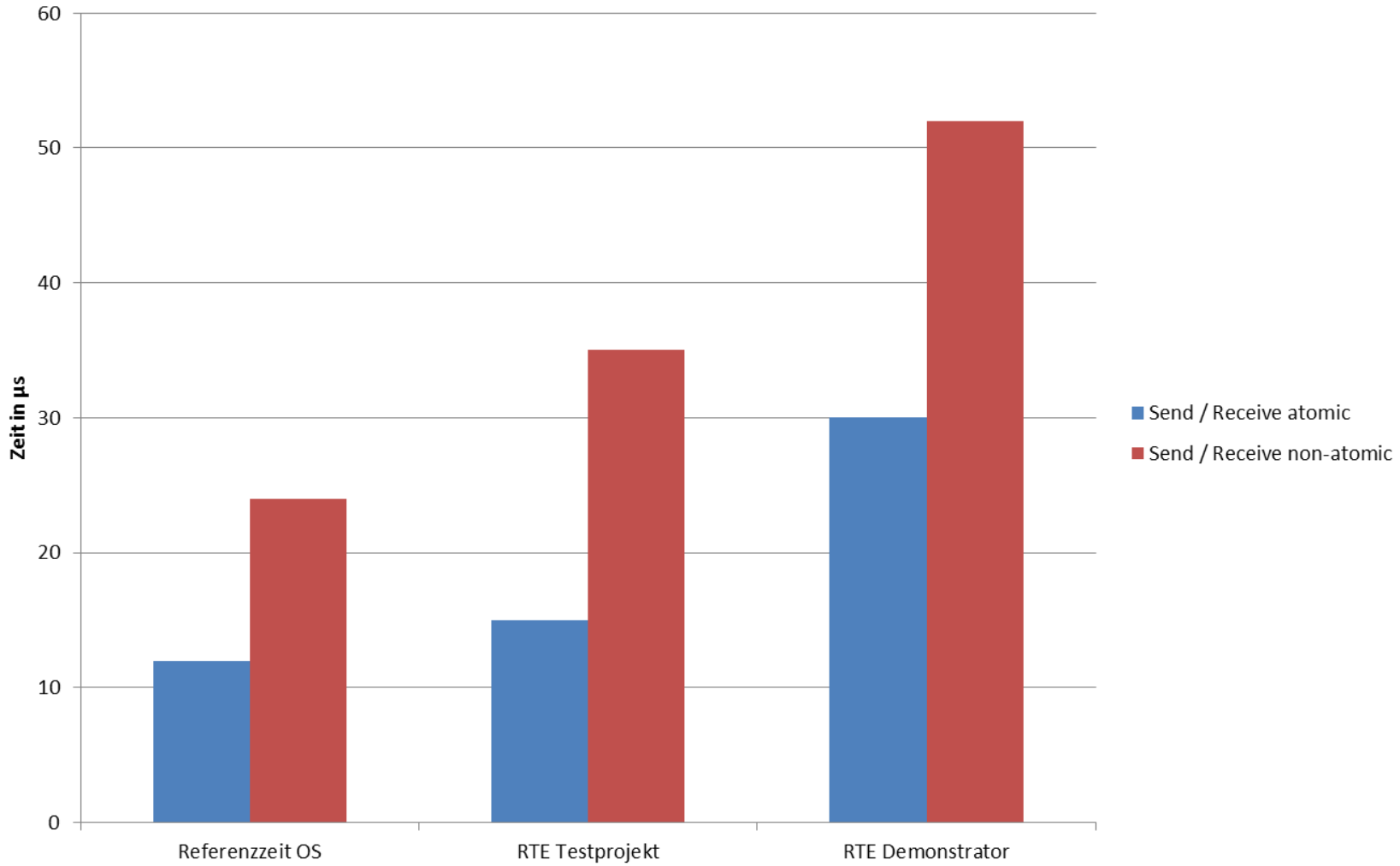
➔ AutosarOS is created automatically by Tresos (sometimes ;-)

Similar process is followed for creating the COM stack.

Integration – Theory and Real Life



Another issue: timing



Validation of the Autosar approach

Pro

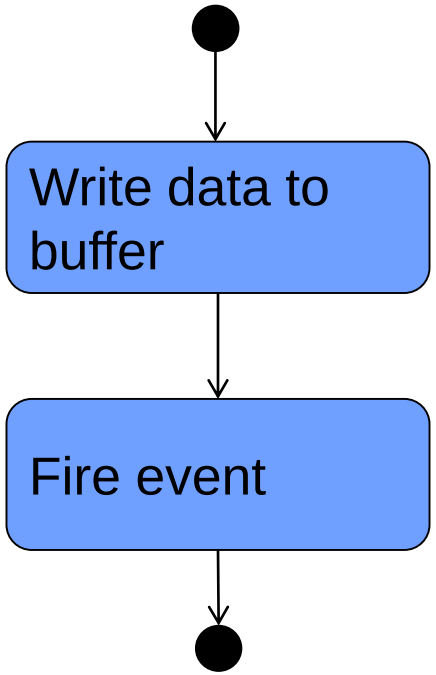
- Good separation of components (loose coupling)
- Easy reuse

Con

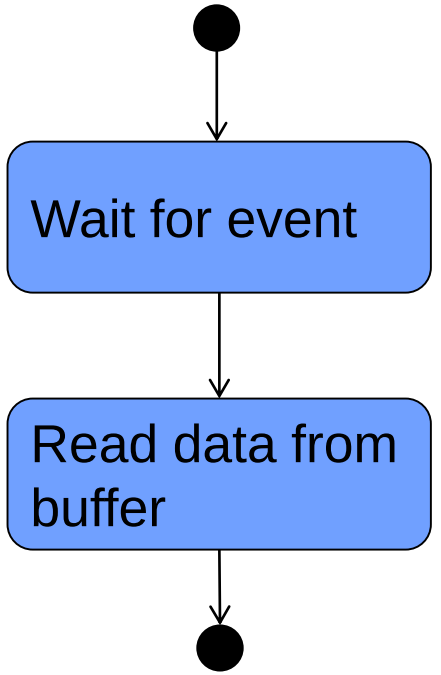
- Complex tooling
- Bad timing
- Hard to debug generated code

RTE General Recipe for Sender/Receiver

Sending data



Receiving data



Simplified concept – RTE Light

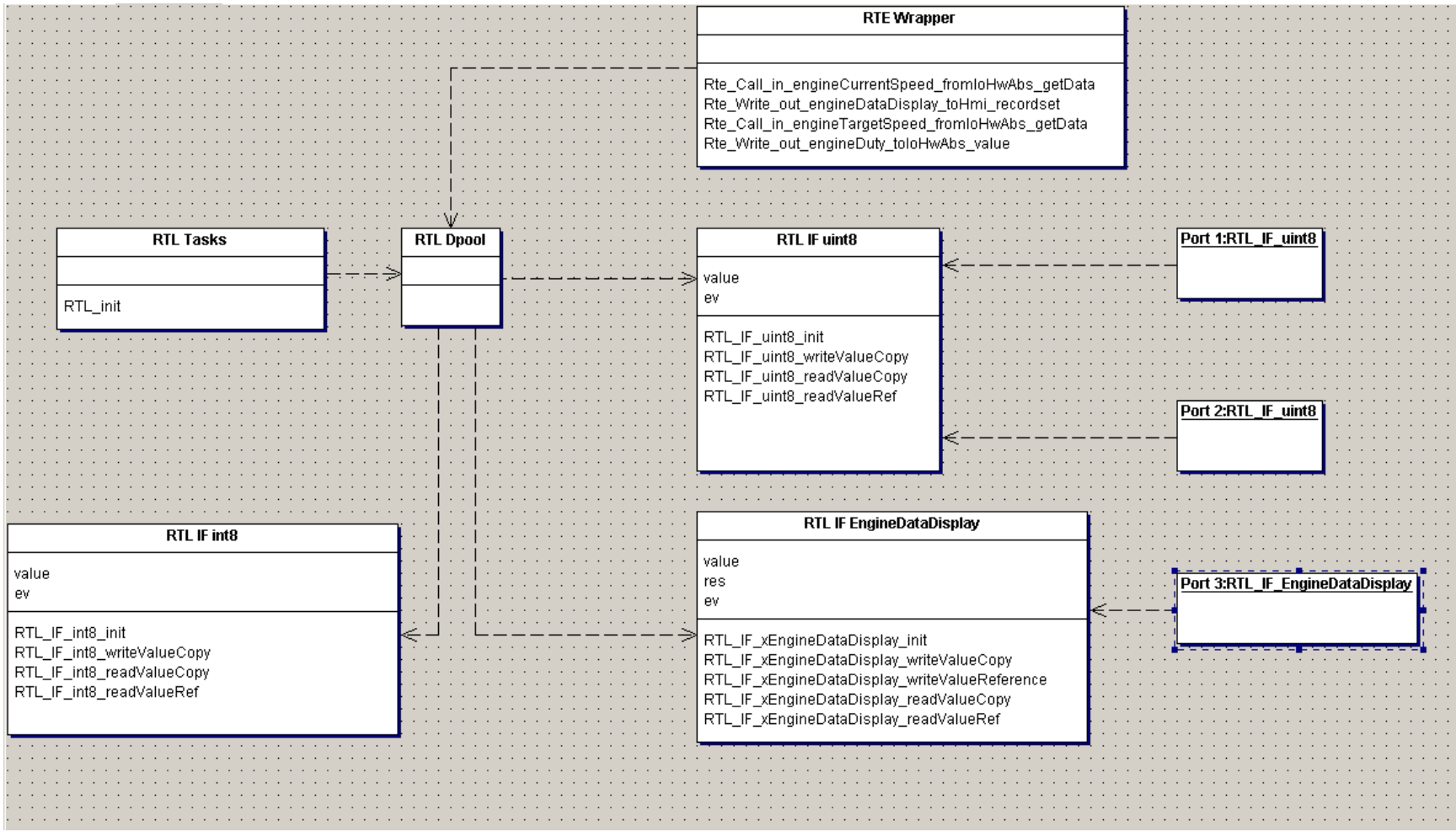
Design goal

- Using the Autosar philosophy for hardware abstraction and loose coupling

- Support of Autosar components as well as legacy code
- Integration of MCAL driver as well as own driver

- Configuration of the system in the code, without the need for complex tooling
- Improved performace and better debugging

RTE Light Architecture



RTE Light
Access to data pool

Instead of using a central data pool, every port gets an own structure.

- data element(s)
- event
- ressource

```
void RTL_IF_uint8_writeValueCopy(IF_uint8_t* me,
                                 uint8_t const value)
{
   //Start of critical section
   //GetResource(me->res); //only needed for non atomic
   me->value = value;
   //ReleaseResource(me->res);

   //Fire Event
   if (me->ev != 0)
  {
      SetEvent(tsk_event,me->ev);
  }
}
```

RTE Light
Configuration of runnables – init / deinit

```
//Function table for init functions
RTL_pFct_t RTL_initFunctions[] = {IOHW_init, SEC_init, EC_init, HMI_init};

//Function table for de-init functions
RTL_pFct_t RTL_deInitFunctions[] = {SEC_deInit, EC_deInit, HMI_deInit};
```

RTE Light
Configuration of runnables – cyclic runnables

```
//struct for cyclic runnable
typedef struct{
        RTL_pFct_t       runnable;
        uint32_t         cycleTime;
        boolean_t        criticalSection;
} RTL_cyclicRunnable_t;

//Function table for cyclic runnables
//Runnable name, time, critical
volatile static const RTL_cyclicRunnable_t RTL_cyclicHighPrioFunction[]  = {{0}};

volatile static const RTL_cyclicRunnable_t RTL_cyclicMediumPrioFunction[] = {
        {EC_run10ms, 10, TRUE},
};

volatile static const RTL_cyclicRunnable_t RTL_cyclicLowPrioFunction[] = {
        {HMI_run10ms, 10, FALSE}
};
```

RTE Light
Configuration of runnables – event triggered runnables

```
//Declare events and ressources for every port
DeclareEvent(ev_writePort_engineDuty_fromEcToIo);
DeclareEvent(ev_writePort_displayValue_fromHmiToIo);
DeclareEvent(ev_writePort_engineDataDisplay_fromEcToHmi);
DeclareEvent(ev_writePort_engineEmergency_fromSecToEc);
DeclareEvent(ev_isr_hmiModeSwitch_fromIsrToHmi);


DeclareResource(res_Port_EngineDiagnostic);
DeclareResource(res_Port_engineDataDisplay);
//All other ports are atomic



//Add one entry for every event firing a runnable
RTL_portTrigger_t RTL_portTriggerSet[] = {
        {ev_writePort_engineDuty_fromEcToIo, IOHW_runEv},
        {ev_writePort_displayValue_fromHmiToIo, IOHW_runEv},
        {ev_writePort_engineDataDisplay_fromEcToHmi, HMI_runEv},
        {ev_isr_hmiModeSwitch_fromIsrToHmi, HMI_runEv},
        {ev_writePort_engineEmergency_fromSecToEc, EC_runEv}
};
```

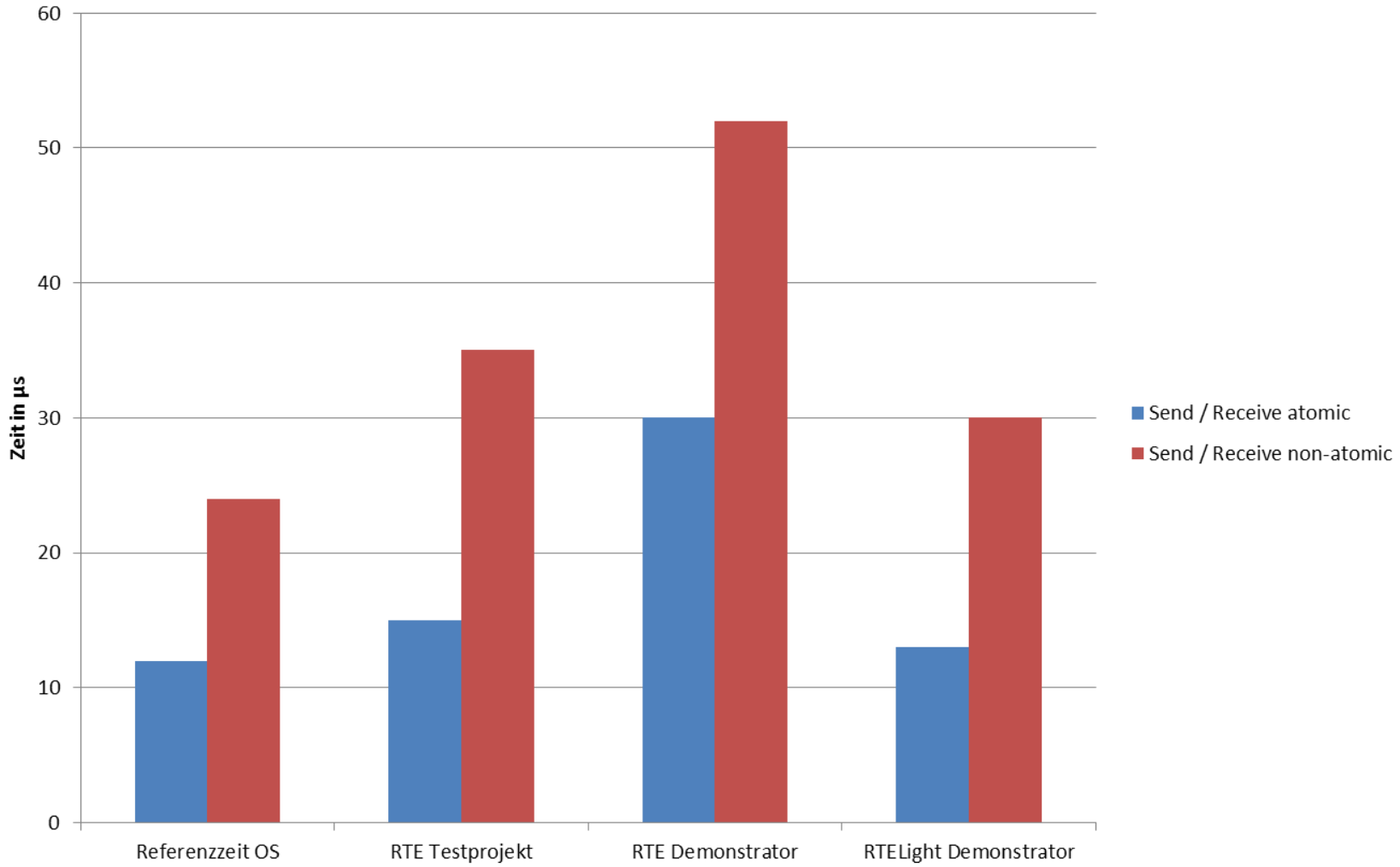
RTE Light
Code for firing the cyclic runnables

```
TASK(tsk_cyclicHighPrio)
{
...
//Call runnables
for (loop = 0; loop < noRunnables; ++loop)
{
        if ((RTL_cyclicHighPrioFunction[loop].runnable != 0) &&
            (count % RTL_cyclicHighPrioFunction[loop].cycleTime) == 0)
        {
          //Check if critical section needs to be secured
          if (RTL_cyclicHighPrioFunction[loop].criticalSection == TRUE)
              GetResource(res_cyclicTask);

          //Call runnable
          RTL_cyclicHighPrioFunction[loop].runnable();

          //Release resource
          if (RTL_cyclicHighPrioFunction[loop].criticalSection == TRUE)
              ReleaseResource(res_cyclicTask);
          }
        }
        count += 10; //Increment by cycletime - static
        TerminateTask();
}
```

RTE Light
Code for firing the event based runnables

```
TASK(tsk_event)
{
//Calculate EventMask out of event data structure
for (loop = 0; loop < noEventRunnables; ++loop)
        eventmask |= RTL_portTriggerSet[loop].ev;
...
while (1)
{
        WaitEvent(eventmask);
        GetEvent(tsk_event,&eventcurrent);

        for (loop = 0; loop < noEventRunnables); ++loop)
        {
           if (RTL_portTriggerSet[loop].ev & eventcurrent)
           {
             //Fitting event has been fired
             ClearEvent(RTL_portTriggerSet[loop].ev);
             //Call the runnable
              RTL_portTriggerSet[loop].runnable(RTL_portTriggerSet[loop].ev);
            }
        }
}
TerminateTask();
}
```

RTE Light
Latency



Wrap-Up: Autosar Meta Model